Apunte teórico - Lab. Algoritmos y Estructura de Datos

ÍNDICE:

1. ¿Qué es programar?
2. Estructura básica de un programa en C
3. Variables
4. Tipos de datos
5. Entrada y salida de datos (printf y scanf)
6. Operadores
7. Incremento de variables
8. Operadores de comparación
9. Condicionales (if, else)
10. Operadores lógicos (&&, ||, !)
11. Booleanos (bool, true, false)
12. Funciones
13. Bucles (while, for)

---

1. ¿QUÉ ES PROGRAMAR?

Definición:

Programar consiste en escribir instrucciones para que una computadora realice una tarea determinada.

En palabras simples:

Programar es enseñarle a una computadora qué hacer paso a paso.

- La computadora no piensa.
- No interpreta.
- No supone.

Hace exactamente lo que le indicamos.

Si le damos instrucciones incorrectas, hará algo incorrecto.

Ejemplo:

Queremos calcular el doble de un número.

La computadora no sabe qué significa "doble".

Debemos decirle:

```
int numero = 5;
int resultado = numero * 2;
```

---

2. ESTRUCTURA BÁSICA DE UN PROGRAMA EN C

```
#include <stdio.h>

int main()
```

```
{
  printf("Hola mundo");

  return 0;
}
```

¿Qué es #include?

Permite incorporar bibliotecas al programa.

Las bibliotecas contienen funciones que ya fueron creadas por otros programadores.

Ejemplo

#include <stdio.h>

Permite utilizar:

printf()

scanf()

#include <stdbool.h>

Permite utilizar:

bool

true

false

¿Qué es main()?

La función main es el punto de entrada del programa.

Cuando ejecutamos un programa, la primera función que se ejecuta es:

int main()

Podemos pensarla como:

La puerta principal del programa.

Todo comienza ahi.

¿Qué significa return 0?

Indica que el programa terminó correctamente.

Por ahora podemos pensar:

Todo programa en C termina con:

return 0;

---

3. VARIABLES

Definición:

Una variable es un “espacio” para almacenar información.

En palabras simples:

Podemos pensar una variable como una “cajita” donde guardamos datos.

Cada “cajita“ tiene:

- una etiqueta (nombre)

- un tipo específico de dato (qué cosa almacena/guarda)

Ejemplo cotidiano:
Imaginemos varias cajas:
Caja "edad"
Caja "altura"
Caja "nombre"

Cada una guarda algo distinto, **y yo no puedo tratar de almacenar una “altura” en la caja “edad”.**

Declaración: En C una variable se “crea” de la siguiente forma:
tipo_dato nombre_variable;

Ejemplos
int edad;
float altura;
char inicial;

Declaración con asignación
int edad = 16;

¿Qué significa el "="?
En programación:
edad = 16;

significa:
Guardar el valor 16 dentro de la variable (caja) edad.
NO significa igualdad matemática. (Comparacion)
Para comparar usamos doble igual (==)
edad == 16 → el valor que almacena edad es igual a 16? (lease en forma de pregunta)

---

4. TIPOS DE DATOS
Definición
Los tipos de datos determinan qué clase de información puede almacenar una variable, o sea, que cosa puedo almacenar en cada “caja”:

Int → Almacena números enteros. (-10, -3, 0, 2, 7, etc…)
int edad = 16;

Float → Almacena números con decimales. (-3.5, 2.7, 4.8, etc..)

NOTACIÓN: en programación los números decimales no se escriben con coma (,) como en matemática, sino en su lugar con un punto (.)

1.5 NO es lo mismo que 1,5

float altura = 1.75;

Char → Almacena un único carácter.

char letra = 'A';

Bool → Almacena verdadero o falso. (True o False)

bool aprobado = true;

---

5. ENTRADA Y SALIDA DE DATOS

Definición:

Permiten que el usuario interactúe con el programa.

printf()

Muestra información por pantalla. (output)

printf("Hola mundo");

scanf()

Permite ingresar información. (input)

scanf("%d", &edad);

¿Qué significa %d?

Le indica a scanf que recibirá un entero.

%d → int

%f → float

%c → char

¿Qué significa el &?

Indica qué variable usaremos (y donde se almacena esa variable)

Por ahora podemos pensar:

Siempre que usemos scanf con variables numéricas debemos colocar &.

---

6. OPERADORES ARITMETICOS:

+ → Suma (3 + 5 = 8)

- → Resta (3 - 5 = -2)

* → Mutiplicacion (3 * 5 = 15)

/ → División entera (7 / 2 = 3), es decir descarta el resto

% → Módulo (Resto de la división, lo que sobra) (7 % 2 = 1)

Ejemplo
15 % 7

15 ÷ 7 = 2
sobra 1

Resultado:
1

Divisibilidad
Si:
numero % divisor == 0

entonces el número es divisible.
Ejemplos:
14 % 7 = 0 (no hay resto en esta división)
20 % 2 = 0 (no hay resto en esta división)
15 % 7 = 1 (**NO** son divisibles, porque hay resto)

---

7. INCREMENTO DE VARIABLES
Muchas veces necesitamos aumentar una variable.
Ejemplo:
edad = edad + 1;
Si edad vale:
15
ahora valdrá:
16

Forma abreviada
edad += 1;
Hace exactamente lo mismo.

Más ejemplos
puntos += 10;
equivale a:
puntos = puntos + 10;
Si puntos almacenaba el valor 5, ahora puntos almacena 15.

Incremento de 1
Existe una forma aún más corta:
edad++;
equivale a:
edad = edad + 1;

---

8. OPERADORES DE COMPARACIÓN
Igual → ==
Pregunta: ¿Son iguales?
Ejemplo:
edad == 18
Distinto → !=
Pregunta: ¿Son distintos?
Ejemplo:
edad != 18
Mayor → >
Ejemplo:
edad > 18

Menor → <
Ejemplo:

edad < 18

Mayor o igual → >=

Menor o igual → <=

---

9. CONDICIONALES
Definición
Permiten tomar decisiones.

Ejemplo cotidiano:
Si llueve:
llevo paraguas.
Si no llueve:
no llevo paraguas.

Sintaxis
if(condicion)

```
{
   instrucciones;
}
```

Ejemplo

```
if(edad >= 18)
{
   printf("Es mayor");
}
```

Explicacion:

El if evalua si el valor que almacena la variable edad es mayor o igual a 18, esta condición devuelve:

- True en caso de cumplirse → imprime “Es mayor”
- False en caso de que no se cumpla → no hace nada (en este caso)

```
if(edad >= 18)
{
   printf("Mayor");
}
else
{
   printf("Menor");
}
```

Acá entra en juego el “else”, esto se lee:

Si (condición), muestro “mayor”, si no (o sea, si no se cumple) muestro “menor”

El if evalúa si el valor que almacena la variable edad es mayor o igual a 18, esta condición devuelve:

- True en caso de cumplirse → imprime “Es mayor”
- False en caso de que no se cumpla → imprime “Menor”

Entonces: si pasa esto, hace esto. Si no paso eso, hace esto

Así se “leería” coloquialmente.

---

10. OPERADORES LÓGICOS

Pensemos los operadores lógicos como “conectores” de condiciones, estos de alguna forma UNEN condiciones y determinan qué valor de verdad tienen (True o False)

AND (&&)

Todas las condiciones deben cumplirse.

if(edad >= 18 && estado == True)

Condición 1: edad mayor o igual a 18

Condición 2: estado igual a True

Para que **TODA** la condición sea verdadera, la condición 1 y 2 deben ser **VERDADERAS** (True)

OR (||)

Al menos una condición debe cumplirse.

if(categoria == 2 || categoria == 7)

Si la categoría es dos **o** la categoría es 7 → si alguna de las dos se cumple, **TODA** la condición es **VERDADERA**

NOT (!)

Invierte el resultado.

!true

Resultado:

false

---

11. BOOLEANOS

Definición:

Representan valores lógicos.

Solo existen dos posibilidades:

1. True
2. False

Ejemplo:

bool aprobado = true;

---

12. FUNCIONES

Definición:

Una función es un bloque de código que realiza una tarea específica.

Ejemplo cotidiano:

Pensá en una licuadora.

Ingredientes → parámetros

Proceso → código
Resultado → return

Sintaxis:

```
tipo_retorno nombre(parametros)
{
   codigo;
}
```

Tipo de retorno es el TIPO DE DATO que la función va a devolver (retornar), si quiero que devuelva un ENTERO, seria int

Ejemplo

```
int doble(int numero)
{
   return numero * 2;
}
```

Partes de una función

```
int doble(int numero)
{
   return numero * 2;
}
```

Tipo de retorno:
int

Nombre:
doble

Parámetro:
numero (de tipo int)

Retorno:
return numero * 2;

Llamar una función

```
int resultado = doble(5);
```

Resultado:
10

GUARDO en una variable de tipo int llamada resultado, al resultado que devuelve la función doble cuando le ingresa el valor 5.
Esto seria hacer 5 * 2 → 10

13.1. BUCLE WHILE
Definición
Repite instrucciones mientras una condición sea verdadera.

Ejemplo

```
int i = 1;

while(i <= 5)
{
    printf("%d\n", i);
    i++;
}
```

i empieza con valor = 1, mientras que i sea menor o igual a 5, vamos a printear el valor de i
Y además incrementamos i en 1, que quiere decir eso? En cada iteración que hace el while, a la variable i se le va a sumar un 1.
Entonces:
1er iteracion: i = 1
El while pregunta: i es menor o igual que 5? Si! Es menor, entonces
Printea i (1)
Y se incrementa i + 1 → 1 + 1 = 2

Entonces repite este proceso hasta que la condición del while (i menor o igual a 5) deje de cumplirse, osea se repetirá hasta que i sea = 6, ya que cuando i vale 1,2,3,4,5 la condición de que sea menor O igual a 5 es VERDADERA

13.2. BUCLE FOR
Definición
Permite repetir una cantidad conocida de veces.

Ejemplo

```
for(int i = 1; i <= 5; i++)
{
```

```
    printf("%d\n", i);
}
```

Aca el bucle se va a repetir con la cantidad de iteraciones que nosotros le dígamos, en este caso itera 5 veces

i = 1, i = 2, i = 3, i = 4, i = 5 (ultima)
